

Bernáth Tamás

Felhasználói dokumentáció

A program egy telefonkönyvkezelő alkalmazást valósít meg. A felhasználó tárolhatja, kezelheti a kontaktjait.

Négyféle különböző típust lehet tárolni: Alap, Üzleti, Diák és Nyugdíjas. Vannak adatok, melyek mind-egyikben megtalálhatók, ilyenek a vezetéknév, keresztnév és becenév. Ezek mellett el lehet tárolni a kontakt címét, egy munkahelyi és egy privát telefonszámot. Az Alap típusú kontakt eltárolja ezek mellett a kontakt telefonjának márkáját, az Üzleti képes eltárolni még egy címet, melynek munkahelyi cím a neve, a Diák típus képes eltárolni a személy iskolájának nevét és évfolyamát végül a Nyugdíjas eltárolja az adott személy nyugdíjazásának évét és a klubtagságát.

Amikor elindítjuk a programot, akkor a főmenüben találjuk magunkat, ahonnan a billentyűzeten található számokkal tudunk haladni, visszalépni, mozogni a programon belül. Fontos, hogy ezeket a számokat használjuk, mert különben nem fogunk tudni haladni. Illetve ha számok helyett betűket használunk a menüpontok elérésére, akkor az ötödik próbálkozás után kidob minket a program. Ha megnyomjuk az 1-est, akkor kilistázza nekünk a program a konzolra a már meglévő kontaktok adatait. Ha ez megtörtént, újra a főmenüben leszünk.

Ha a 2-est nyomjuk meg, akkor a rendezés menüben leszünk, ahol a következők szerint rendezhetjük a kontakjainkat: vezetéknév az 1-es, becenév a 2-es és város a 3-as billentyű lenyomásával. Ha lerendeztük, akkor már nem állíthatjuk vissza az eredeti állapotot. Rendezés után a konzolon nem jelenik meg semmi új, továbbra is a rendezés menüt látjuk, a 4-es billentyű lenyomásával tudunk visszalépni a főmenübe.

A főmenüben a 3-ast lenyomva a kezelés menübe jutunk. Itt az 1-es billentyű lenyomásával új kontaktokat vehetünk fel, ha lenyomjuk akkor egy új menübe kerülünk, ahol kiválaszthatjuk hogy milyen típusú személyt szeretnénk felvenni, 1: Alap, 2: Üzleti, 3: Diák, 4: Nyugdíjas. Miután itt is választottunk nekiállhatunk bevinni a kontakt adatait. Ezt soronként tehetjük meg. A program kiírja, hogy milyen adatot vár éppen. Miután beírtuk, az Enter lenyomásával vihetjük be az adatot. Ha egy sorban nem írunk semmit csak Entert nyomunk, azt semmisnek veszi a program, azaz nem számít bevitt adatnak az üres sor. Miután a program nem vár több bemenetet, az utolsó Enter után a program hozzáadja az új kontaktot az eddigiekhez és kilép az előző menüpontba, ahol azt választhatjuk, hogy milyen adatot viszünk fel. A kezelés menüben még választhatjuk az adatok index (2), vagy név(3) általi törlését. Ha nincs ilyen indexű elem (az indexelés 0-tól indul) vagy nincs ilyen nevű kontaktunk tárolva, akkor a program kiírja nekünk a konzolra majd újra behozza a kezelés menüt. Ebben a menüben még módosítani van

lehetőségünk. Ha a 4-est lenyomjuk, először ki kell választanunk, hogy melyik indexű elemet szeretnénk módosítani, utána a már megszokott módon vehetjük fel a módosított kontakt adatait. A kezelés menüből az 5-ös gomb lenyomásával léphetünk vissza a főmenübe.

A főmenüben a 4-es billentyű lenyomásával érhetjük el a szűrés menüt. Itt szűrhetünk vezetéknév, becenév vagy város szerint. Ezekhez ebben a sorrendben az 1, 2 vagy 3-s billentyűt kell lenyomni. Miután kiválasztottuk, hogy mi alapján szeretnénk szűrni, a program kilistázza nekünk a szűrésnek megfelelő kontaktokat vagy ha nincs ilyen, akkor kiírja, hogy nincs ilyen kontakt. Innen a 4-es billentyűvel léphetünk vissza a főmenübe.

A főmenüből az 5-ös gomb lenyomásával léphetünk a keresés menübe. Itt kiválaszthatjuk, hogy mi szerint szeretnénk keresni, név vagy telefonszám szerint. Ha talál ilyen személyt, akkor a program kilistázza az adatait illetve megmondja, hogy hányadik indexű elem. A hármas billentyűvel léphetünk vissza a főmenübe.

A fájlkezelés menüt a főmenüből a 6-os gomb lenyomásával érjük el. Itt választhatunk, hogy az összes felvett kontaktunkat kiírjuk egy előre meghatározott nevű txt fájlba vagy pedig betöltjük ugyanabból a fájlból a kontaktokat. Ha a fájlkezelés műveletek sikertelenek hibaüzenetet kapunk, ha nem kapunk üzenetet az azt jelenti, hogy sikeres volt a művelet. Ebből az almenüből a 3-as gomb lenyomásával léphetünk ki.

A programból pedig úgy tudunk kilépni, hogy a főmenüben megnyomjuk a 7-es billentyűt. Ekkor a program bezáródik.

Bernáth Tamás

Telefonkönyv – programozói dokumentáció

A program egy telefonkönyvkezelő alkalmazást valósít meg

A program főbb elemei:

- A kontakt osztályok (Alap, Diak, Nyugdíjas, Uzleti)
- A menükezelő függvények (fomenu, kezeles_menu, rendez_menu, fajlkezelesmenu)
- Egy generikus tároló (sajatvektor.hpp)
- Az adattárolást segítő adatszerkezetek: Cím, Telefonszam, Nev

Adatszerkezetek:

Cím:

Egy osztály, ami attribútumaiban eltárol kettő intet: egy irszámot és egy házszámot, kettő std::stringet: várost és utcanévet. Az osztály rendelkezik mindegyik attribútumhoz setter és getter függvényekkel. Illetve tud kiírni tetszőleges streamre és konzolra is.

Telefonszám:

Ez az osztály két std::string típusú adattagot tárol: munkahelyi szám és privát szám. Az osztály biztosít getter és setter metódusokat a fenti attribútumokra. Képes fájlba írni, illetve beolvasni a számokat, valamint túlterhelt << operátor segítségével megjeleníthető a konzolon vagy fájlban.

Név:

A név kezelésére szolgáló osztály, amely három std::string adattagot tartalmaz: vezetéknév, keresztnév, becenév. Az osztály rendelkezik mindegyik attribútumhoz setter és getter függvényekkel. Illetve tud kiírni tetszőleges streamre és konzolra is.

Kontakt osztályok:

Személy (absztrakt) osztály:

Ez az összes programban szereplő kontakt osztály közös ősoosztálya. Tartalmazza az általános adatokat, amit mindegyik öröklött osztály is (név, cím, telefonszám) és az ezekhez tartozó setter-getter függvényeket. Illetve 3 tisztán virtuális függvényt: kiír, beolvas, fajlbair. Ezeket a függvényeket átírjuk a későbbi osztályokban.

Alap osztály

Az alap osztály a Szemely osztály származtatott osztálya. Legalapvetőbb kontakt, egy extra attribútumot tárol: marka. Ez egy string és a személy telefonmárkáját tárolja. Felülírja a Szemely virtuális függvényeit, ezzel biztosítja a megfelelő kiírást, beolvasást és fájlba írást. Az osztály ugyanazokat a műveleteket tudja, mint a Szemely típus, ezért minden olyan helyen használható, ahol Szemely* típusú adatot vár a program például a telefonkönyv tárolójában is gond nélkül kezelhető.

Diak osztály

Ez az osztály a Szemely származtatott osztálya. Két extra attribútumot tárol: az iskola nevét (string) és az évfolyamot (int). Felülírja a Szemely virtuális függvényeit, ezért itt is működik a konzolra/fájlba írás illetve a beolvasás. Az osztály ugyanazokat a műveleteket tudja, mint a Szemely típus, ezért minden olyan helyen használható, ahol Szemely* típusú adatot vár a program, például a telefonkönyv tárolójában is gond nélkül kezelhető.

Uzleti osztály

Ez az osztály a Szemely osztály származtatott osztálya. Egy extra attribútummal rendelkezik: egy extra üzleti címmel (Cim). Felülírja a Szemely virtuális függvényeit, így itt is működik a helyes kiírás, beolvasás. Az osztály ugyanazokat a műveleteket tudja, mint a Szemely típus, ezért minden olyan helyen használható, ahol Szemely* típusú adatot vár a program, például a telefonkönyv tárolójában is gond nélkül kezelhető.

Nyugdíjas osztály

Ez az osztály is a Szemely osztály származtatott osztálya. Két extra attribútuma: klubtagság(bool), illetve a nyugdíjazás éve (int). Felülírja a Szemely virtuális függvényeit, a kiírás, beolvasás itt is helyesen működik. Az osztály ugyanazokat a műveleteket tudja, mint a Szemely típus, ezért minden olyan helyen használható, ahol Szemely* típusú adatot vár a program, például a telefonkönyv tárolójában is gond nélkül kezelhető.

Ezek az osztályok a Szemely absztrakt ősosztályból származnak, és közös tulajdonságokat (név, cím, telefonszám) örökölnek. A Szemely osztály is rendelkezik minden attribútumához tartozó setter és getter függvényekkel. Ezek mellett 3 virtuális függvénnyel:

- virtual void kiir(ostream& os) const = 0;
- virtual void beolvas(std::istream& is) = 0;
- virtual void fajlba_ir(std::ostream& os) = 0;

Ezek mind felül lesznek írva a származtatott osztályokban.

A Szemely ősosztályból 4 további osztály származik: Alap, Diak, Nyugdíjas, Uzleti.

Alap:

```
string Alap::getMarka() const;
```

A márkát lehet vele lekérdezni

```
void Alap::setMarka(const string& tmp);
```

A márkát lehet vele beállítani.

```
void Alap::kiir(ostream& os) const;
```

A konzolra történő szép kiírásért felel, felülír egy virtuális függvényt.

```
void Alap::beolvas(std::istream& is);
```

A beolvasásért felel, tetszőleges streamről, az adatokat sortörések választják el bevitelkor, felülír egy virtuális függvényt.

```
ostream& operator<<(ostream& os, const Alap& tmp);
```

```
istream& operator>>(istream& is, Alap& tmp);
```

operátor túlterhelések

```
void Alap::fajlba_ir(std::ostream& os);
```

Fájlbaírásért felelő függvény, az adatokat úgy viszi be egy fájlba, hogy onnan azt a beolvas függvény be is tudja olvasni.

Diak:

```
string Diak::getIskola() const
```

```
int Diak::getEvfolyam() const
```

```
void Diak::setEvfolyam(int tmp
```

```
void Diak::setIskola(const string& tmp)
```

A Diak osztály saját attribútumainak setter, getter függvényei

```
void Diak::kiir(ostream& os) const;
```

A konzolra történő szép kiírásért felel, felülír egy virtuális függvényt.

```
void Diak::beolvas(std::istream& is);
```

A beolvasásért felel, tetszőleges streamről, az adatokat sortörések választják el bevitelkor, felülír egy virtuális függvényt.

```
ostream& operator<<(ostream& os, const Alap& tmp);
```

```
istream& operator>>(istream& is, Alap& tmp);
```

operátor túlterhelések

```
void Diak::fajlba_ir(std::ostream& os);
```

Fájlbaírásért felelő függvény, az adatokat úgy viszi be egy fájlba, hogy onnan azt a beolvas függvény be is tudja olvasni.

Nyugdíjas:

```
bool Nyugdijas::getKlubtag() const;
```

```
int Nyugdijas::getNyugdijazas() const;
```

```
void Nyugdijas::setNyugdijazas(int tmp);
```

```
void Nyugdijas::setKlubtag(bool tmp);
```

setter getter függvények

Ebben az osztályban is megtalálhatóak a kiír és beolvas függvények felülírásai illetve az operátor túlterhelések. Ezek szinte teljesen megegyeznek az eddig leírtakkal ezért nem részletezem őket.

Uzleti:

```
Cim Uzleti::getMunkahelyiCim() const;
```

```
void Uzleti::setMunkahelyiCim(int irszam, const string& varos, const string& utca, int hazzsam);
```

setter getter függvények

Ebben az osztályban is megtalálhatóak a kiír és beolvas függvények felülírásai illetve az operátor túlterhelések. Ezek szinte teljesen megegyeznek az addig leírtakkal ezért nem részletezem őket.

SajátVektor:

A SajátVektor egy sablonos osztály, ami dinamikus tömbként működik. Egy STL vector-hoz hasonló működésű, de nem használ külső könyvtárakat. A tároló automatikusan bővül, ha betelik. Lehet a benne lévő elemeket törölni illetve módosítani. Az osztály rendelkezik egy getter függvénnyel, ami visszaad egy mutatót, ami az elemek tömbjének elejére mutat. A SajátVektor osztály általánosan használható bármilyen típushoz, ezért a telefonkönyv is ezt használja a különböző típusú személyek (Szemely*) tárolására. Ez biztosítja, hogy a különböző kontakt típusok dinamikusan és hatékonyan legyenek kezelhetők a programban.

A SajátVektor osztály egy sablonos dinamikus tömb.

```
void Felvesz(T elem);
```

Elemet vesz fel a tárolóba, ha nincs elég hely, akkor előtte bővíti a kapacitást

```
void Torol(int index);
```

Index alapján töröl elemet a tárolóból

```
void Modosit(int index, T uj)
```

Meglévő elemet lehet módosítani, úgy csinálja, hogy törlni az adott elemet és egy újat szúr be a helyére

```
T* getElements() const;
```

Visszaadja az első elemre mutató pointert

```
int getMeret() const;
```

Visszaadja a tároló méretét

Telefonkönyv

Ez az osztály a program központi része. Ez felel a kontaktok eléréséért. Egy SajátVektor tárolót használ, ami Szemely* típusú elemeket tartalmaz, így képes kezelni az összes, Szemely-ből származtatott osztályt. Az osztály biztosítja az adatok felvételét, törlését (név vagy index alapján), módosítását, listázását, keresését, rendezését és szűrését.

```
void Felvesz(Szemely* s);
```

Felvesz egy Szemely*-t a tárolóba

```
void Telefonkonyv::Torol_index();
```

Index alapján töröl egy elemet a tárolóból, a törlésért a SajatVektor törlő függvénye felel

```
void Telefonkonyv::Torol_nev();
```

Név alapján töröl elemet, a nevet nem paraméterben kell megadni, hanem a függvény kéri be futása közben. A törlésért itt is a SajatVektor törlő függvénye felel.

```
void Telefonkonyv::ListazOsszes(std::ostream& os) const;
```

Tetszőleges streamre kilistázza az összes kontaktot

```
void Telefonkonyv::KeresNev();
```

Név alapján keres a tárolóban, ha talál egyezést kiírja az adatait.

```
void Telefonkonyv::KeresSzam() const;
```

Szám alapján keres a tárolóban

```
void Telefonkonyv::Rendez(const std::string& szempont);
```

Ez a függvény a telefonkönyvben tárolt személyeket rendezi megadott szempont alapján. A rendezés során egyszerű buborékredezés algoritmust használ.

```
void Telefonkonyv::SzuresAlapjan(const std::string& mi, std::ostream& os) const;
```

A telefonkönyv bejegyzéseit egy adott szűrési feltétel alapján kiírja a megadott kimeneti adatfolyamba.

A felhasználótól bekéri az összehasonlítandó értéket, majd minden kontakt esetén ellenőrzi, hogy az megfelel-e a megadott szempontnak.

```
void Telefonkonyv::AlapFelvesz();
```

Alap típusú elemet vesz fel a tárolótömbbe

```
void Telefonkonyv::UzletiFelvesz();
```

Uzleti típusú elemet vesz fel a tárolótömbbe

```
void Telefonkonyv::NyugdijasFelvesz();
```

Nyugdijas típusú elemet vesz fel a tárolótömbbe

```
void Telefonkonyv::Modosit();
```

A függvény egy meglévő kontaktot lecserél egy új, felhasználó által megadott típusú és adatú személlyel.

Fájlbaírás:

Ez az osztály felel a fájlkezelésért

void Fajlbairas(Telefonkonyv& konyv);

A telefonkönyv összes bejegyzését kiírja egy fájlba, típusmegjelöléssel együtt, felülírva a meglévő tartalmat.

void Fejlbeolvasas(Telefonkonyv& konyv);

Egy fájlból beolvassa és újraépíti a telefonkönyv tartalmát a soronként megadott típusok alapján.

A különböző menükezelők:

A program sok azonos működésű menükezelőt használ, amiknek közel azonos a paraméterlistája. Mindegyik bekér egy Telefonkönyv referenciát és két tetszőleges streamet. A függvények futásuk közben várják a bemeneteket. Mindegyik belsejében egy switch-case található, amikben a konzolra kiírt menüpontok segítenek tájékozódni. Az egyetlen eltérő menükezelő a fájlkezelésmenu modul, ami a paraméterlistájába még vár egy Fajlkezeles referenciát.